

REPORT PRACTICE 2

OPERATING SYSTEMS

Denis Loren Moldovan 100522240@alumnos.uc3m.es	100522240
Antonio Nicolas Lemus Yeguas 100522110@alumnos.uc3m.es	100522110
Jorge Adrian Saghin Dudulea 100522257@alumnos.uc3m.es	100522257

INDEX

Code Description.....	2
scripter.c.....	2
mygrep.c.....	2
Test cases.....	3
Conclusion.....	4

Code Description

The directory structure of the lab was divided into /src, /build and /tests, for better management and modularity. The Makefile was initially written to compile all source files from /src into /build, and ran from there, using the InputScripts from /tests. The explanation for the whole project is commented inside the source code. The following will be used as an overview of the functionality of the lab.

scripter.c

Apart from the initial functions, we have added a struct to store all the information for each command inside each line. This would hold the arguments inside an array, the number of said arguments, the PID of the process executing that command (whose functionality was finally dropped), and the redirection file of STDERR. We have also developed new functions, used for modularity inside the code:

- *tokenizar_linea* (initial code): Modified to take an undefined number of tokens.
- *procesar_redirecciones* (initial code): Modified so the loop doesn't go out of bounds, and added a condition to iterate through the commands correctly.
- *command_pipes*: Function used to close and duplicate the file descriptors from the pipes accordingly.
- *procesar_linea* (initial code): Modified to process the required functionality.
- *parse_file*: Used to retrieve all the contents of the provided file, and parse it into multiple lines.

mygrep.c

As the file was practically empty, we developed both functions, and the algorithm used. Because the program was not that complex, we could complete it with just two functions, both called from the main function:

- *check_contains*: Checks if the input string is inside the buffer of the file, stored as a global variable.
- *parse_file*: Takes as input the name of the file and populates the buffer with the contents of it.

Test cases

Test ID	Description	Inputs	Expected outputs
TCs01	Verify basic command execution.	## Script de SSOO ls -l echo "Hello World"	Output of `ls -l`, and "Hello World" printed to stdout.
TCs02	Invalid script header	# Invalid Header ls	Error: Script doesn't follow convention: Invalid argument
TCs03	Empty line in script	## Script de SSOO ls [empty line] echo "test"	Error: Incorrect format of command: Operation not permitted
TCs04	Output redirection (>)	## Script de SSOO echo "Hello" > output.txt cat output.txt	Hello (written to output.txt and printed)
TCs05	Piped commands	## Script de SSOO ls -l grep scripiter wc -l	Prints to stdout the number of files that contain "scripiter"
TCs06	Background execution (&)	## Script de SSOO sleep 5 &	[PID] (printed immediately)
TCs07	Invalid command	## Script de SSOO invalidcommand	Error: execvp failed: No such file or directory
TCm01	String found in file	mygrep testfile.txt "search" (testfile.txt contains "search" twice)	Lines containing "search"
TCm02	String not found	mygrep testfile.txt "missing"	"missing" not found.
TCm03	File not found	mygrep nonexistent.txt "hello"	Error opening reading file: No such file or directory
TCm04	Multiple matches	mygrep log.txt "error" (log.txt has "error" twice)	Lines with "error"
TCm05	Empty file	mygrep empty.txt "anything"	(No output, exits successfully)
TCm06	Case-sensitive search	mygrep testfile.txt "Hello" (File has "hello" and "Hello")	Only lines with "Hello"

Conclusion

As to conclude, the whole lab was not that complicated, as all the concepts were explained during the lectures, and the statement is really complete. The only problem we encountered was a problem with the pipes, where they weren't being closed correctly, therefore, the child process would enter into a deadlock state and the parent process wouldn't continue. This took a lot of debugging time, and we finally found out after looking at examples online, and trying them out in our own code.

This was a really interesting laboratory, because we were able to interact directly with the `fork()` function in Linux, and this was also an opportunity to learn about how to use C more in depth.